Московский физико-технический институт
Физтех-школа прикладной математики и информатики

ОСНОВЫ КРИПТОГРАФИИ

V СЕМЕСТР

Лектор: *Даниил Владимирович Мусатов*

$h\nu$

Автор: *Киселев Николай*
Репозиторий на Github

осень 2026

Содержание

1 Введение **2**

1.1 Квантовая и постквантовая криптографии 2

2 Односторонние функции **2**

2.1 Односторонние перестановки 5

2.2 Односторонние перестановки с секретом 5

3 Генераторы случайных чисел **6**

3.1 Теорема о существовании ГПСЧ из односторонней функции 8

4 Семейства псевдослучайных функций **14**

4.1 Конструкция ПСФ из ГПСЧ 15

5 Задача шифрования **16**

5.1 Модели атак 17

5.1.1 Схемы, надежные относительно одноразового подслушивания 17

5.1.2 Схемы, надежные относительно многоразового подслушивания 18

5.1.3 Атаки с выбором сообщения 18

6 Протоколы привязки **19**

6.1 Неинтерактивный протокол привязки 20

6.2 Интерактивный протокол привязки 21

7 Протоколы идентификации **22**

7.1 Доказательство знания с нулевым разглашением 23

8 Задача аутентификации **23**

9 Цифровая подпись **24**

9.1 Протокол одноразовой подписи 1 бита с открытым ключом 25

9.2 Протоколы одноразовой подписи с открытым ключом 25

1 Введение

Ну короче тыры-пыры все дела там секретно все и тд.

Сначала поймем, какие модели угроз мы будем рассматривать (какие алгоритмы взломщик может запускать). В основной теории разделяют 2 основных вида:

1. Равномерный противник — полиномиальный вероятностный алгоритм. Причем полином берется не по длине отправленного сообщения, а по параметру n, который называется параметром безопасности (например, для 512-битного шифрования $n = 512$).
2. Неравномерный противник — набор схем (например, злоумышленник может их физически спаять).

Напоминание (Теорема Эйдлмана)**.** $\mathbf{BPP} \subset \mathbf{P/poly}$

Таким образом, неравномерный противник сильнее равномерного. Заметим, что если $\mathbf{P} = \mathbf{NP}$, то тогда криптография с открытым ключом не имеет смысла: по открытому ключу можно быстро (за полином) получить закрытый. Однако существуют такие протоколы, которые не требуют неравенства данных двух классов.

В задачах по криптографии обычно Алиса хочет передать сообщение Бобу, а Ева (злоумышленник) пытается взломать шифрование

Пример (Гаммирование (одноразовый блокнот))**.** Пусть Алиса хочет передать m Бобу. В таком случае они (заранее) договариваются о некотором секрете k, таком, что $|k| = |m|$. При передаче сообщения, Алиса отправляет $n = m \oplus k$ (побитово), Боб делает то же самое и получает $m = n \oplus k$.

Далее мы будем жить в предположении, что $\mathbf{P} \neq \mathbf{NP}$.

1.1 Квантовая и постквантовая криптографии

Относительно недавно выяснилось, что квантовые компьютеры умеют раскладывать числа на множители. Важно понимать, что настоящие **NPC**-задачи (по-видимому, FACTORING такой не является) пока что решить при помощи него не получилось. Однако на данный момент вычислительных мощностей даже квантового компьютера не хватает для разложения чисел, используемых в современной криптографии.

Соответственно, постквантовая криптография — это классические протоколы, устойчивые относительно квантового противника. На данный момент главный претендент на такой протокол — теория решеток.

Квантовая криптография — это протокол, использующий квантовый канал связи. Таким образом, если кто-то вдруг будет смотреть за тем, что мы там передаем, то будут помехи и получатель сможет понять, что за каналом следят (далее начинается очень сложная квантовая механика).

Оба данных подхода могут спасти от вмешательства квантового компьютера.

2 Односторонние функции

Интуитивно, односторонняя функция — такая, что $x \mapsto f(x)$ легко вычислить, а $f(x) \mapsto x$ уже сложно. В силу того, что можно положить $f(x) = 1$ для всех x, задача

обращения односторонней функции звучит так: "Для данного y найти хотя бы один x, такой, что $f(x) = y$".

Пусть $f = \{f_n\}_{n=1}^{\infty}$, где $f_n : \{0,1\}^{k(n)} \to \{0,1\}^{l(n)}$ (регулярная по длине, т.е. при равной длине входа получается равная длина выхода). Здесь n — параметр безопасности, k, l — полиномы.

Определение 2.1. f называется слабо односторонней, если $\exists q(\cdot)$ — полином $\forall R(\cdot)$ — обратитель (либо равномерный, либо неравномерный противник) $\exists N \forall n > N : \Pr_{x \in \{0,1\}^{k(n)}}\{R(f(x)) \in f^{-1}(f(x))\} < 1 - \frac{1}{q(n)}$

Определение 2.2. f называется сильно односторонней, если $\forall p(\cdot)$ — полином $\forall R(\cdot)$ — обратитель (либо равномерный, либо неравномерный противник) $\exists N \forall n > N : \Pr_{x \in \{0,1\}^{k(n)}}\{R(f(x)) \in f^{-1}(f(x))\} < \frac{1}{q(n)}$

Замечание. На самом деле, выше написаны 4 определения — по два для равномерного и неравномерного противника

Замечание. Предположительно, функция $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^{2n}, f(x,y) = x \cdot y$ является односторонней.

Теорема 2.1 (Об усилении)**.** *Существует слабо односторонняя функция $\Rightarrow$ существует сильно односторонняя функция.*

Для доказательства теоремы потребуются несколько утверждений:

Утверждение 2.1. *Пусть f — сильно односторонняя функция. Пусть $g(x,y) = (f(x), y)$. Тогда g — сильно односторонняя.*

Доказательство. Очевидно, что g — регулярная по длине полиномиальная функция. Пусть теперь у g есть обратитель R_g. Покажем, что тогда и у f есть обратитель R_f. Положим $R_f(z) = (R_g(z,y))_x$. Далее нетрудно можно посчитать распределение полученных ответов и доказать утверждение. □

Конструкция теоремы об усилении. Рассмотрим функцию $F(x_1, \dots x_M) = (f(x_1), \dots f(x_M))$ (здесь $M = M(n)$). Если обратитель будет наивным и будет обращать вектор покомпонентно, то он угадает с вероятностью $\left(1 - \frac{1}{q(n)}\right)^M$, что можно сделать сколь угодно малым. □

Однако, обратитель вовсе не обязан быть наивным — он может действовать как-то по-другому, необязательно обращая функцию покомпонентно. Для доказательства, мы не будем из обратителя функции f строить обратитель F. Наоборот, из обратителя F мы построим очень хороший обратитель f.

Доказательство теоремы об усилении. Предположим, что F не сильно односторонняя. Тогда

$$\exists q(\cdot) \exists R_f \forall N \exists n > N \Pr\{R(F(x)) \in F^{-1}(F(x))\} > \frac{1}{q(n)}$$

Построим по R_F обратитель R_f функции f. Пусть мы хотим обратить $f^{-1}(y) = \dots$. Дополним y до $(y, f(x_2), \dots f(x_M))$ (заметим, что после такого дополнения полученный вектор имеет такое же распределение, как и x из определения R_F) и рассмотрим $(\vec{z_1}, \dots \vec{z_M}) = R_F(y, f(x_2), \dots f(x_M))$, после чего проверим $f(z_1) = y$. Повторим данную операцию Q раз,

причем сделаем эти Q итераций для каждого места, где может стоять y (проще говоря, поставим y на всевозможные места и для каждого из них проведем данную операцию независимо Q раз). Если мы увидим хотя бы один успех, то возвращаем его. Так строится R_f. Докажем теперь, что он действительно быстро обращает функцию f.

Рассмотрим функцию $S_i(x) = \Pr\left\{\left(R_F(f(x_1), \dots f(x_{i-1}), f(x), f(x_{i+1}), \dots f(x_M))\right)_i \in f^{-1}(f(x))\right\}$ и положим $S(x) = \max\{S_1(x), \dots S_M(x)\}$. Выберем некоторый параметр $\varepsilon \sim \frac{1}{poly(n)}$ (определим его позже) и разобьем слова на простые: для которых $S(x) \geqslant \varepsilon$ и сложные: для которых $S(x) < \varepsilon$. Также положим δ — доля сложных слов.

Лемма 2.1. $\Pr\{R_f \textit{ ошибся}\} \leqslant \delta + (1-\delta)(1-\varepsilon)^Q$

Доказательство.

$$\Pr\{R_f \text{ ошибся}\} \leqslant \underbrace{\Pr\{x \text{ сложное}\}}_{\delta} + \underbrace{\Pr\{x \text{ простое}\}}_{1-\delta} \cdot \underbrace{\Pr\{R_f \text{ ошибся} | x \text{ простое}\}}_{(1-\varepsilon)^Q}$$

□

Лемма 2.2. $\Pr\{R_F \textit{ успешен}\} \leqslant M\delta\varepsilon + (1-\delta)^M$

Доказательство. Покроем все вероятностное пространство событиями:

Событие	Pr
Все x_i — простые	$(1-\delta)^M$
x_1 — простые	δ
$\vdots$	$\vdots$
x_M — простые	δ

Соответственно, по формуле полной вероятности:

$$\Pr\{R_F \text{ успешен}\} \leqslant \Pr\{R_F \text{ успешен} | \text{все } x_i \text{ простые}\} \cdot (1-\delta)^M +$$

$$+ \sum_{i=1}^{M} \Pr\{R_F \text{ успешен} | x_i \text{ сложное}\} \cdot \delta \leqslant M\delta\varepsilon + (1-\delta)^M$$

□

Поскольку мы предположили противное, то $\Pr\{R_f \text{ ошибается}\} \geqslant \frac{1}{p(n)}$ и $\Pr\{R_F \text{ успешен}\} \geqslant \frac{1}{q(n)}$. Подберем параметры M, Q, ε так, чтобы получилось противоречие с доказанными леммами:

1. $Q = \frac{n}{\varepsilon} \Rightarrow (1-\varepsilon)^Q \approx e^{-n} \Rightarrow \delta \geqslant \frac{1}{2p(n)}$
2. $M = np(n) \Rightarrow (1-\delta)^M \sim e^{-\frac{n}{2}} \Rightarrow M\delta\varepsilon \geqslant \frac{1}{2q(n)} \Rightarrow \varepsilon \geqslant \frac{1}{2q(n)M\delta} \geqslant \frac{1}{2q(n)M}$
3. Наконец, положим $\varepsilon = \frac{1}{4q(n)np(n)}$ — получим противоречие с неравенством выше.

□

2.1 Односторонние перестановки

Рассмотрим одностороннюю функцию, являющуюся биекцией, т.е. $f = \{f_n\}_{n=1}^{\infty}$, где $f_n : \{0,1\}^{k(n)} \to \{0,1\}^{k(n)}$.

К сожалению, на данный момент нет даже кандидатов в примеры таких функций (на всём $\{0,1\}^{k(n)}$). В связи с этим рассматриваются более слабые свойства: биекция $f_n : D_n \to D_n$, где $D_n \subset \{0,1\}^n$. То есть мы разрешаем функции быть перестановкой не на всех строках длины n, а только на некотором подмножестве.

Пример (Кандидат в примеры)**.** Рассмотрим функцию Рабина: $(m, x) \mapsto (m, x^2 \mod m)$, где $x \underset{m}{\equiv} y^2$. Для $m = 4k+3$ — простого данная функция будет легко обратимой: достаточно взять $R_f(z) = z^{\frac{m+1}{4}}$. Тогда: $R_f(x^2) = R_f(y^4) = y^2 = x$.

Теперь рассмотрим $m = pq$, где $p, q \underset{4}{\equiv} 3$. Если p, q известны, то обратить функцию легко алгоритмом выше, после чего использовать КТО. В случае же, когда p, q неизвестны, обратить её уже становится сложнее.

Заметим, что в примере выше область определения — это не все строки длины n, а именно некоторое подмножество D_n. Но тут возникает естественный вопрос: а как нам вообще работать с элементами из D_n? Если мы хотим, скажем, выбрать случайный элемент из D_n (чтобы подать его на вход функции), то нам нужно уметь его эффективно сгенерировать. Причём сгенерировать так, чтобы распределение было “почти равномерным” на D_n — иначе противник может использовать неравномерность в свою пользу.

Определение 2.3. В общем определении требуется, чтобы за полиномиальное время можно было сгенерировать распределение, пренебрежимо мало отличимое от равномерного на D_n. Более формально, пусть α — сгенерированное распределение, тогда

$$\forall p(\cdot) \exists N : \forall n > N \forall S \subset D_n \left| \alpha(S) - \frac{|S|}{|D_n|} \right| < \frac{1}{p(n)}$$

2.2 Односторонние перестановки с секретом

Примечание от лектора: по-другому: ”one-way trapdoor permutations”

Обычные односторонние перестановки функция — те, для которых $f_\alpha(x)$ легко вычислима, а вот $f_\alpha^{-1}(y)$ — трудно (если знать только α). Перестановка с секретом — та, для которой $f_\alpha^{-1}(y)$ легко вычисляется, если знать α и некоторый секрет τ. Дадим этому формальное определение:

Определение 2.4. Односторонняя перестановка с секретом — функция $f : \{0,1\}^{k(n)} \times \{0,1\}^{l(n)} \to \{0,1\}^{l(n)}$ (мы будем обозначать $f_\alpha(x) := f(\alpha, x)$), для которой существует множества $D_\alpha \subset \{0,1\}^{l(n)}$, причем $f : D_\alpha \to D_\alpha$ — биекция и 4 полиномиальных вероятностных алгоритма:

1. Генератор (генерирует ключи) G : случаные биты $\mapsto$ пара(α, τ)
2. Сэмплер (сэмплирует D_α) $S : \alpha \mapsto$ почти случайный элемент D_α
3. Вычислитель функции $F : (\alpha, x) \mapsto f_\alpha(x)$
4. Обратитель $B : (\alpha, \tau, y) \mapsto f_\alpha^{-1}(y)$

Кроме этих алгоритмов, еще требуется односторонность перестановки:

$$\forall p \forall R \exists N \forall n > N : \Pr_{\alpha,y}\{R(\alpha, y) = f_\alpha^{-1}(y)\} < \frac{1}{p(n)}$$

Пример (Предположительный пример)**.** Рассмотрим функцию Рабина, где $\alpha = p \cdot q$ (p, q — простые вида $4k+3$) — модуль, по которому производим арифметику, D_α — квадратичные вычеты $\mod \alpha$, секрет — $\tau = (p, q)$, $f_\alpha(x) = x^2 \mod \alpha$.

Замечание. Про данную функцию неизвестно, действительно ли она является необратимой, но точно можно сказать, что квантовым компьютером данный алгоритм можно взломать (а именно, разложить на множители число pq). Таким образом, данная функция является примером **НЕ** постквантовой криптографии.

3 Генераторы случайных чисел

Можно генерировать числа по-разному. Можно рассматривать ГИСЧ — генераторы истинно случайных чисел и ГПСЧ — генераторы псевдо-случайных чисел. Последние почти всегда работают по следующему правилу: дается некоторый x малой длины, по которому строится $G(x)$ большой длины. Нетрудно понять, что таким образом получить равномерное распределение на $\{0,1\}^{|G(x)|}$ нельзя (т.к. $|G(x)| > |x|$), но можно попытаться получить нечто "похожее" на случаное распределение.

Определение 3.1. Статистическое расстояние между распределениями случайных величин α_n, β_n определяется как

$$\operatorname{dist}(\alpha_n, \beta_n) = \max_{S \subset \{0,1\}^n} |\Pr\{\alpha_n \in S\} - \Pr\{\beta_n \in S\}|$$

Утверждение 3.1.

$$\operatorname{dist}(\alpha_n, \beta_n) = \frac{1}{2} \sum_{x \in \{0,1\}^n} |\Pr\{\alpha_n = x\} - \Pr\{\beta_n = x\}|$$

Доказательство. Пусть максимум в определении достигается на множестве S. Тогда он достигается на множестве $\overline{S} = \{0,1\}^n \setminus S$. Б.О.О, $\Pr\{\alpha_n \in S\} \geqslant \Pr\{\beta_n \in S\}$. Тогда $\forall x \in S : \Pr\{\alpha_n = x\} \geqslant \Pr\{\beta_n = x\}$ (иначе можно "убрать" невыгодный x и значение только увеличится). Аналогично $\forall x \in \overline{S} : \Pr\{\alpha_n = x\} \leqslant \Pr\{\beta_n = x\}$. Заметим тогда, что

$$\Pr\{\alpha_n \in S\} - \Pr\{\beta_n \in S\} = \sum_{x \in S} \Pr\{\alpha_n = x\} - \Pr\{\beta_n = x\} = \sum_{x \in S} |\Pr\{\alpha_n = x\} - \Pr\{\beta_n = x\}|$$

Аналогично:

$$\Pr\{\alpha_n \in \overline{S}\} - \Pr\{\beta_n \in \overline{S}\} = \sum_{x \in S} \Pr\{\beta_n = x\} - \Pr\{\alpha_n = x\} = \sum_{x \in \overline{S}} |\Pr\{\alpha_n = x\} - \Pr\{\beta_n = x\}|$$

Но тогда:

$$\operatorname{dist}(\alpha_n, \beta_n) = \frac{(\Pr\{\alpha_n \in S\} - \Pr\{\beta_n \in S\}) + \left(\Pr\{\beta_n \in \overline{S}\} - \Pr\{\alpha_n \in \overline{S}\}\right)}{2} =$$

$$= \frac{1}{2} \sum_{x \in \{0,1\}^n} |\Pr\{\alpha_n = x\} - \Pr\{\beta_n = x\}|$$

□

Замечание. Таким образом, dist является метрикой (L_1-метрикой) на множестве всех случайных величин.

Определение 3.2. Будем говорить, что α_n, β_n статистически близки ($\alpha_n \sim \beta_n$), если

$$\forall p(\cdot) \exists N \forall n > N : \operatorname{dist}(\alpha_n, \beta_n) < \frac{1}{p(n)}$$

Замечание. Из утверждения выше получается эквивалентное определение:

$$\alpha_n \sim \beta_n \Leftrightarrow \forall p(\cdot) \forall T \exists N \forall n > N : |\Pr\{T(\alpha_n) = 1\} - \Pr\{T(\beta_n) = 1\}| < \frac{1}{p(n)}$$

Здесь T — любая булева функция

Определение 3.3. Будем говорить, что α_n, β_n вычислительно неотличимы ($\alpha_n \sim \beta_n$, будет понятно из контекста, что имеется в виду), если

$$\forall p(\cdot) \forall T \exists N \forall n > N : |\Pr\{T(\alpha_n) = 1\} - \Pr\{T(\beta_n) = 1\}| < \frac{1}{p(n)}$$

Где T — противник (полиномиально вычислимый вероятностный алгоритм или семейство схем полиномиальной длины).

Замечание. Статистическая близость и вычислительная неотличимость — отношения эквивалентности

Доказательство. Рефлексивность и симметричность очевидны. Докажем транзитивность: рассмотрим моменты N_1, N_2, для которых $\forall n > N_1, N_2 : \operatorname{dist}(\alpha_n, \beta_n) < \frac{1}{2p(n)}, \operatorname{dist}(\beta_n, \gamma_n) < \frac{1}{2p(n)}$. Но тогда $\forall n > \max\{N_1, N_2\} : \operatorname{dist}(\alpha_n, \gamma_n) < \frac{1}{p(n)}$ □

Утверждение 3.2. *Пусть $\alpha_n \sim \beta_n$ (вычислительно неотличимы) и f — полиномиально вычислимая функция. Тогда $f(\alpha_n) \sim f(\beta_n)$ — тоже вычислимо неотличимы.*

Доказательство. Пусть $\exists p \exists T \forall N \exists n > N : |\Pr\{T(f(\alpha_n)) = 1\} - \Pr\{T(f(\beta_n)) = 1\}| \geqslant \frac{1}{p(n)}$. Рассмотрим $T' = T \circ f$ — полиномиально вычислимый. Но тогда $\forall N \exists n > N : |\Pr\{T'(\alpha_n) = 1\} - \Pr\{T'(\beta_n) = 1\}| \geqslant \frac{1}{p(n)}$, получили, что $\alpha_n \not\sim \beta_n$. □

Определение 3.4. Генератор псевдо-случайных чисел — полиномиально вычислимая функция (детерминированным алгоритмом) $G_n : \{0,1\}^n \to \{0,1\}^{p(n)}, p(n) > n$, такая, что $G(x) \sim y$, где y — случайные биты длины $p(n)$ (при условии, что $|x| = n$). Противник для ГПСЧ обычно называют отличителем.

Замечание. Обычно конструкция ГПСЧ выглядит следующим образом: имеется $g : \{0,1\}^n \to \{0,1\}^{n+1}$. Соответственно $g(x) = (y, b)$ (b — последний бит). Соответственно, вывод определяется следующим образом: выдаем последний бит и передаем y обратно на вход функции g, так делаем сколько угодно раз.

Теорема 3.1. $\exists$ *ГПСЧ* $\Rightarrow \exists$ *односторонняя функция*

Доказательство. Докажем, что сам генератор будет являться односторонней функцией. Пусть R — обратитель для G. Тогда для бесконечно многих n: $\Pr_{x\in\{0,1\}^n}\{R(G(x)) \in G^{-1}(G(x))\} = \varepsilon(n) \geqslant \frac{1}{q(n)}$. Рассмотрим функцию

$$T(y) = \begin{cases} 1, G(R(y)) = y \\ 0, \text{иначе} \end{cases}$$

Заметим, что $|G(\{0,1\}^n)| \leqslant 2^{n-1}$. В таком случае:

$$\Pr\{T(y) = 1\} = \begin{cases} \varepsilon, y \text{ из распределения } G(x) \\ \frac{|G(\{0,1\}^n)|}{2^n} \leqslant \frac{\varepsilon}{2}, y \text{ — случайный} \end{cases}$$

Но тогда $\left|\Pr_{y\in\{0,1\}^{p(n)}}\{T(y)=1\} - \Pr_{x\in\{0,1\}^n}\{T(G(x))=1\}\right| \geqslant \frac{\varepsilon(n)}{2} \geqslant \frac{1}{2q(n)}$, поэтому $G(x) \not\sim y$, получили противоречие. □

В книжке Верещагина и Шеня можно найти доказательство следующей сложной теореме:

Теорема 3.2 (б/д)**.** $\exists$ *односторонняя функция* $\Rightarrow$ *существует ГПСЧ*

Мы докажем следующее более слабое утверждение:

3.1 Теорема о существовании ГПСЧ из односторонней функции

В данном разделе будем под односторонней перестановкой понимать перестановку с областью определения $\{0,1\}^n$

Определение 3.5. Пусть $h : \{0,1\}^n \to \{0,1\}, g : \{0,1\}^n \to \{0,1\}^n$ — полиномиально вычислимые функции. Функция h называется трудным битом для функции g, если

$$\forall q \forall P \exists N \forall n > N \left|\Pr\{P(g(x)) = h(x)\} - \frac{1}{2}\right| \leqslant \frac{1}{q(n)}$$

P обычно называют предсказателем

Замечание. В частности, функция g должна быть труднообратимой.

Теорема 3.3. $\exists$ *односторонняя перестановка* $\Rightarrow \forall p(n) \exists$ *ГПСЧ* $G : \{0,1\}^n \to \{0,1\}^{p(n)}$.

Схема доказательства. Пусть $f : \{0,1\}^n \to \{0,1\}^n$ — односторонняя перестановка. По ней мы получим односторонюю перестановку с трудным битом, а именно: $g(x,y) = (f(x), y)$, а $h(x,y) = x \odot y = \sum_{i=1}^n x_i y_i (\mathrm{mod}\, 2)$. После этого получим $G(x) = g(x)h(x)$, и общий случай $G'(x) = h(x)h(g(x))\dots h(g^{p(n)-1}(x))$ — что и будет являться ГПСЧ для данного полинома. □

Замечание. По схеме доказательства можно заметить следующее: если мы прочитаем у первые n бит выхода построенного ГПСЧ G, то не сможем (это будет трудно) вычислить оставшийся бит.

Для доказательств пунктов теоремы потребуются некоторые дополнительные факты:

Определение 3.6. $\mathrm{WH} : \{0,1\}^n \to \{0,1\}^{2^n}$ определяется следующим образом: $\mathrm{WH}(x) = (x \odot y)_{y \in \{0,1\}^n}$ — т.е. это вектор из всех скалярные произведения x на векторы из $\{0,1\}^n$.

Замечание. По сути, $\mathrm{WH}(x)$ — таблица истинности для $f_x(y) = x \odot y \mod 2$

Замечание.
1. $f(s \oplus r) = f(s) \oplus f(r)$
2. $x \neq 0 \Rightarrow$ в $\mathrm{WH}(x)$ поровну 0 и 1.
3. $x \neq z \Rightarrow \mathrm{WH}(x)$ совпадает с $\mathrm{WH}(z)$ в половине координат

Пусть теперь у нас есть настоящий код $w = \mathrm{WH}(x)$ и $\hat{w}$ — строка, отличающаяся от w в доле позиций $< \varepsilon < \frac{1}{4}$. Но тогда по $\hat{w}$ можно однозначно восстановить w.

Напоминание. Пусть ξ_i — попарно независимые бернуллевские случайные величины со значениями в $\{0,1\}$, такие что $\mathbb{E}\xi_i \geqslant \frac{1}{2} + \varepsilon$. Тогда для их суммы выполнено неравенство Чебышёва:

$$\Pr\left\{\sum_{i=1}^{N} \xi_i \leqslant \frac{N}{2}\right\} < \frac{1}{N\varepsilon^2}$$

Теорема 3.4 (б/д)**.** *В любом шаре радиуса $\frac{1}{2} - \frac{1}{poly(n)}$ (по метрике количества различных позиций) лежит $poly(n)$ кодовых слов Уолша-Адамара*

Теорема 3.5 (Левина-Голдрайха)**.** *Пусть $f : \{0,1\}^n \to \{0,1\}^n$ — односторонняя перестановка. Тогда функция $g(x,y) = (f(x), y)$ является односторонней перестановкой, а функция $h(x,y) = x \odot y$ является трудным битом для нее.*

Доказательство. Предположим противное: функция $h(x,y) = x \odot y$ не является трудным битом для односторонней перестановки $g(x,y) = (f(x), y)$. По определению это означает, что существует полиномиальный предсказатель P (для определенности рассмотрим неравномерного противника — последовательность схем полиномиального размера), который для бесконечно многих n угадывает значение $h(x,y)$ с заметным преимуществом:

$$\Pr_{x,y \in \{0,1\}^n}\{P(f(x), y) = x \odot y\} \geqslant \frac{1}{2} + \varepsilon(n),$$

где $\varepsilon(n) \geqslant \frac{1}{q(n)}$ для некоторого многочлена q. Наша цель — построить полиномиальный инвертор R_f, который, используя этот предсказатель P, сможет эффективно находить прообраз x по данному значению $f(x)$, что опровергнет односторонность исходной перестановки f.

Применим стандартный вероятностный метод усреднения. Назовем строку $x \in \{0,1\}^n$ *хорошей*, если предсказатель дает правильный ответ на случайных направлениях y с вероятностью не менее $\frac{1}{2} + \frac{\varepsilon(n)}{2}$. Несложно показать, что доля хороших строк среди всех возможных входов длины n составляет не менее $\frac{\varepsilon(n)}{2}$. В дальнейшем мы опишем алгоритм восстановления x именно для таких хороших строк.

Для наглядности зафиксируем хорошую строку x и положим, что нам дан её образ $f(x)$. Обозначим через $A(y) := P(f(x), y)$ функцию, вычисляемую предсказателем на этом фиксированном образе. По определению хорошей строки:

$$\Pr_y\{A(y) = x \odot y\} \geqslant \frac{1}{2} + \frac{\varepsilon(n)}{2}.$$

В терминах кодов Уолша-Адамара строка A длины 2^n (таблица истинности предсказателя) совпадает с истинным кодовым словом $w = \mathrm{WH}(x)$ в доле позиций, равной не менее $\frac{1}{2} + \frac{\varepsilon(n)}{2}$.

Реализуем алгоритм восстановления пошагово, опираясь на ключевые идеи из набросков:

1. **Сведение к локальному декодированию списка.** Поскольку доля ошибок в таблице A близка к $\frac{1}{2}$, мы не можем применить обычное однозначное декодирование (которое требует долю ошибок строго менее $\frac{1}{4}$). Однако, согласно теореме выше, в данном шаре радиуса $\frac{1}{2} - \frac{\varepsilon(n)}{2}$ лежит не более $poly(n)$ кодовых слов Уолша-Адамара. Следовательно, исходная задача гарантированно сводится к поиску в полиномиальном списке кандидатов, содержащем истинное слово w.

2. **Метод самокоррекции.** Если бы мы хотели узнать бит $x \odot y$ для конкретного направления y, мы могли бы воспользоваться линейностью скалярного произведения: $x \odot y = (x \odot (y \oplus r)) \oplus (x \odot r)$. Если заменить истинные значения на предсказания алгоритма A, мы получим формулу голосования:

$$\text{Голос}_r(y) = A(y \oplus r) \oplus A(r).$$

   Если бы A ошибался с вероятностью меньше $\frac{1}{4}$, мы могли бы просто взять мажоритарное голосование по случайным r, поскольку обе точки $y \oplus r$ и r одновременно оказались бы правильными с вероятностью большей $\frac{1}{2}$. Но в нашем случае вероятность ошибки предсказателя близка к $\frac{1}{2}$, поэтому значение $A(r)$ слишком часто оказывается неверным.

3. **Попарно независимая генерация точек.** Чтобы снизить влияние ошибок, выберем небольшое число $k = O(\log n)$ *абсолютно случайных* и независимых векторов $u_1, \ldots, u_k \in \{0,1\}^n$. Для каждого непустого подмножества индексов $I \subset \{1, \ldots, k\}$ сформируем вектор $r_I = \bigoplus_{i \in I} u_i$. Такое семейство векторов $\{r_I\}$ состоит из $N = 2^k - 1 = poly(n)$ элементов, которые являются *попарно независимыми* случайными величинами, каждая из которых распределена равномерно.

4. **Полиномиальный перебор истинных значений.** Главная идея заключается в том, что мы не знаем правильных значений скалярных произведений $x \odot u_i$. Однако, поскольку k мало ($O(\log n)$), мы можем просто перебрать все возможные варианты значений $x \odot u_i$. Всего таких комбинаций будет $2^k = poly(n)$. Для правильной угаданной комбинации битов мы автоматически знаем точные правильные значения $x \odot r_I$ для всех N векторов благодаря линейности: $x \odot r_I = \bigoplus_{i \in I} (x \odot u_i)$.

Окончательный алгоритм инвертора $R_f(f(x))$:

1. Выбрать случайные векторы $u_1, \ldots, u_k \in \{0,1\}^n$ для $k = \lceil \log_2(4n/\varepsilon^2 + 1) \rceil$. Генерировать попарно независимые r_I.

2. Для каждого из 2^k возможных векторов догадок $\vec{\sigma} = (\sigma_1, \ldots, \sigma_k) \in \{0,1\}^k$ (где гипотетически $\sigma_i = x \odot u_i$):

   (a) Вычислить для всех подмножеств I значение $\sigma_I = \bigoplus_{i \in I} \sigma_i$.

(b) Для каждого базисного вектора $e_j = (0, \dots, 1, \dots, 0)$ (где единица стоит на j-м месте) восстановить j-й бит строки x с помощью мажоритарного выбора:

$$\tilde{x}_j = \mathrm{maj}_I \left\{ P(f(x), e_j \oplus r_I) \oplus \sigma_I \right\}.$$

(c) Сформировать полную строку-кандидат $\tilde{x} = (\tilde{x}_1, \dots, \tilde{x}_n)$.

(d) Проверить условие $f(\tilde{x}) = f(x)$. Если оно выполнено — немедленно вернуть $\tilde{x}$ и завершить работу.

Анализ корректности: Рассмотрим итерацию перебора, на которой вектор догадок оказался абсолютно верным (то есть $\sigma_I = x \odot r_I$). Обозначим через ξ_I indicator события, что предсказание $P(f(x), e_j \oplus r_I)$ совпало с истинным значением $x \odot (e_j \oplus r_I)$. Так как x — хорошая строка, а $e_j \oplus r_I$ распределен равномерно, математическое ожидание $\mathbb{E}\xi_I = \Pr\{P(f(x), e_j \oplus r_I) = x \odot (e_j \oplus r_I)\} \geqslant \frac{1}{2} + \frac{\varepsilon}{2}$.

Поскольку величины r_I попарно независимы, индикаторы ξ_I также попарно независимы. Применим закон Чебышёва из нашего напоминания:

$$\Pr\left\{ \sum_I \xi_I \leqslant \frac{N}{2} \right\} < \frac{1}{N\varepsilon^2}.$$

Поскольку мы выбрали $N = poly(n)$ таким, что $\frac{1}{N\varepsilon^2} < \frac{1}{2n}$, вероятность того, что мажоритарный выбор для конкретного бита $\tilde{x}_j$ ошибется, строго меньше $\frac{1}{2n}$. Тогда, вероятность того, что хотя бы один из n битов вектора x будет восстановлен неверно, не превышает $n \cdot \frac{1}{2n} = \frac{1}{2}$.

Таким образом, для любой хорошей строки алгоритм успешно находит прообраз x с вероятностью не менее $\frac{1}{2}$. С учетом того, что доля хороших строк равна $\frac{\varepsilon(n)}{2}$, суммарная вероятность успеха инвертора R_f составляет не менее $\frac{\varepsilon(n)}{4} \geqslant \frac{1}{poly(n)}$. Поскольку весь перебор и проверки занимают строго полиномиальное время, мы получили противоречие с тем, что перестановка f является односторонней. Теорема доказана. □

Лемма 3.1 (XOR-Лемма Яо)**.** *Пусть g — односторонняя перестановка с трудным битом h. Тогда $G(x) = g(x)h(x)$ является ГПСЧ*

Доказательство. Пусть D — отличитель для G. Покажем, как по нему построить P — предсказатель для трудного бита h. По определению отличителя (запишем явно отрицание к вычислительной неотличимости):

$$\exists q \forall N \exists n > N \left| \Pr\{D(g(x)h(x)) = 1\} - \Pr\{D(yr) = 1\} \right| > \frac{1}{q(n)}$$

Заметим, что g — односторонняя перестановка с областью определения $\{0,1\}^n$, поэтому $x \sim \mathcal{U}(\{0,1\}^n) \Rightarrow g(x) \sim \mathcal{U}(\{0,1\}^n)$. Поэтому данное условие можно переписать так:

$$\exists q \forall N \exists n > N \left| \Pr\{D(g(x)h(x)) = 1\} - \Pr\{D(g(x)r) = 1\} \right| > \frac{1}{q(n)}$$

Положим $P(y) = D(yr) \oplus r \oplus 1$, где r — случайный бит. Тогда значения P определяются

по следующей таблице:

$D(g(x)1)\to$ / $D(g(x)0)\downarrow$	0	1
0	случайно	1
1	0	случайно

Также положим $\alpha, \beta, \gamma, \delta$ — вероятности попадения в клетки, т.е.:

$D(g(x)1)\to$ / $D(g(x)0)\downarrow$	0	1
0	α	β
1	γ	δ

Обозначим $\alpha = \alpha_0 + \alpha_1$ — вероятности событий $h(x) = 0, h(x) = 1$ внутри соответствующей клетки (например, $\beta_0 = P(h(x) = 0, D(g(x)0) = 0, D(g(x)1) = 1)$). Заметим, что:

$$\Pr\{D(g(x)h(x)) = 1\} = \delta_0 + \delta_1 + \beta_1 + \gamma_0$$

$$\Pr\{D(yr) = 1\} = \delta_0 + \delta_1 + \frac{\beta_0 + \beta_1}{2} + \frac{\gamma_0 + \gamma_1}{2}$$

Соответственно:

$$\Pr\{D(g(x)h(x)) = 1\} - \Pr\{D(yr) = 1\} = \delta_0 + \delta_1 + \beta_1 + \gamma_0 - \delta_0 - \delta_1 - \frac{\beta_0 + \beta_1}{2} - \frac{\gamma_0 + \gamma_1}{2} =$$

$$= \frac{\beta_1 - \beta_0}{2} + \frac{\gamma_1 - \gamma_0}{2} > \varepsilon$$

Тогда:

$$\Pr\{P(g(x)) = h(x)\} = \frac{\alpha_0 + \alpha_1}{2} + \frac{\delta_0 + \delta_1}{2} + \beta_1 + \gamma_0 =$$

$$\frac{\alpha_0 + \alpha_1}{2} + \frac{\beta_0 + \beta_1}{2} + \frac{\gamma_0 + \gamma_1}{2} + \frac{\delta_0 + \delta_1}{2} + \frac{\beta_1 - \beta_0}{2} + \frac{\gamma_1 - \gamma_0}{2} > \frac{1}{2} + \varepsilon$$

Получили, что h — не случайный бит для g. □

Лемма 3.2 (Блюма-Микали)**.** *Пусть есть ГПСЧ:* $G : \{0,1\}^n \to \{0,1\}^{n+1}$. *Тогда существует ГПСЧ:* $G' : \{0,1\}^n \to \{0,1\}^{p(n)}$.

Доказательство. Положим $G(x) = g(x)h(x)$, где $g : \{0,1\}^n \to \{0,1\}^n, h : \{0,1\}^n \to \{0,1\}$ — полиномиально вычислимые функции (проще говоря, h — последний бит G, а g — его префикс). Определим $G_2(x) = G(G(x))h(x) = g(g(x))h(g(x))h(x)$. Положим $G_0(zr_1r_2) = zr_1r_2, G_1(yr) = g(y)h(y)r$. Заметим, что $G_0 \sim G_1$ (в смысле вычислительной неотличимости), так как G — генератор, т.е. $g(y)h(y) \sim zr_1$. Рассмотрим преобразование $F(zr_1r_2) = g(z)h(z)r_1$. Заметим, что $G_1(yr) = F(G_0(yrr_2))$, а $G_2(x) = F(G_1(xr))$. Так как F полиномиально вычислима, то $G_1 \sim G_2$, получаем $G_0 \sim G_2$, т.е. G_2 — ГПСЧ.

Покажем теперь, как эта идея с промежуточными распределениями обобщается на случай произвольной длины $p(n)$. Положим $m(n) = p(n) - n$. Для каждого i от 0 до $m(n)$ определим промежуточные случайные величины H_i длины $p(n)$, где первые i битов генерируются последовательными применениями функций h и g, затем идут n битов состояния, а остаток заполняется случайными битами.

Пусть $x \in \{0,1\}^n$ and случайный хвост $r \in \{0,1\}^{m(n)}$ выбираются равномерно. Тогда

наши промежуточные распределения задаются как:

$$H_i = h(x)h(g(x))\dots h(g^{i-1}(x))g^i(x)r_{i+1}\dots r_{m(n)}$$

Крайние распределения в точности соответствуют нужным нам величинам: $H_0 = xr_1\dots r_{m(n)}$ является абсолютно случайным распределением на $\{0,1\}^{p(n)}$ (так как $g^0(x) = x$), а $H_{m(n)}$ — это в точности выход нашего полного генератора $G'(x) = h(x)h(g(x))\dots h(g^{m(n)-1}(x))g^{m(n)}(x)$.

Предположим противное: G' не является надежным генератором. Тогда $H_0 \not\sim H_{m(n)}$, то есть существует полиномиальный отличитель D и многочлен q, такие что для бесконечно многих n:

$$\left|\Pr\{D(H_{m(n)}) = 1\} - \Pr\{D(H_0) = 1\}\right| > \frac{1}{q(n)}$$

Распишем эту разность по неравенству треугольника через соседние шаги:

$$\sum_{i=0}^{m(n)-1} |\Pr\{D(H_{i+1}) = 1\} - \Pr\{D(H_i) = 1\}| \geqslant \left|\Pr\{D(H_{m(n)}) = 1\} - \Pr\{D(H_0) = 1\}\right| > \frac{1}{q(n)}$$

Значит, найдется такой индекс i, для которого отличитель разделяет соседние распределения с непренебрежимой вероятностью:

$$|\Pr\{D(H_{i+1}) = 1\} - \Pr\{D(H_i) = 1\}| > \frac{1}{m(n)q(n)}$$

Построим на основе D эффективный отличитель D' для исходного генератора $G : \{0,1\}^n \to \{0,1\}^{n+1}$. На вход D' подается строка w, которая имеет вид либо $G(z) = g(z)h(z)$, либо является абсолютно случайной строкой $u \cdot b$, где b — случайный бит. Обозначим $w = state \cdot bit$. Алгоритм D' действует следующим образом:

1. Используя значение $state$, восстанавливает префикс промежуточной строки в обратном направлении: полагает $x_i = state$ и для j от $i-1$ до 0 вычисляет бит $b_j = h(x_{j+1})$ и состояние $x_j = g(x_{j+1})$.
2. На $(i+1)$-е место ставит бит bit из входной строки.
3. Оставшуюся часть строки заполняет случайными битами $r_{i+2}\dots r_{m(n)}$.
4. Запускает отличитель D на полученной строке $b_0\dots b_{i-1} \cdot bit \cdot x_0 r_{i+2}\dots r_{m(n)}$ и возвращает его результат.

Если на вход D' пришла псевдослучайная строка $G(z) = g(z)h(z)$, то $state = g(z)$ и $bit = h(z)$. В силу биективности перестановки g, элемент z однозначно равен $g^{-1}(state)$. Восстановленные состояния будут равны $x_j = g^j(z)$, а биты префикса — $b_j = h(g^j(z))$. Полученная строка имеет в точности распределение случайной величины H_{i+1}.

Если же на вход D' пришла случайная строка $u \cdot b$, то построенная строка будет иметь распределение величины H_i, так как u играет роль случайного состояния на i-м шаге, а b — роль случайного бита на $(i+1)$-м месте.

Таким образом, созданный алгоритм D' разделяет распределения исходного генератора G с преимуществом:

$$|\Pr\{D'(G(z)) = 1\} - \Pr\{D'(U_{n+1}) = 1\}| = |\Pr\{D(H_{i+1}) = 1\} - \Pr\{D(H_i) = 1\}| > \frac{1}{m(n)q(n)}$$

Для завершения доказательства разберем два случая в зависимости от типа противника:

- ▷ **Неравномерный противник:** Индекс i зашивается прямо в схему отличителя D'. Полученная схема имеет полиномиальный размер и нарушает надежность исходного генератора G.
- ▷ **Равномерный противник:** Алгоритм D' на первом шаге выбирает индекс $i \in \{0, \dots, m(n) - 1\}$ случайно и равномерно. Тогда его преимущество составит не менее $\frac{1}{m(n)} \cdot \frac{1}{m(n)q(n)} = \frac{1}{m^2(n)q(n)}$, что не является пренебрежимо малым и приводит к противоречию с надежностью генератора G.

Лемма доказана. □

Замечание. Собрав все утверждения вместе, получаем доказательство исходной теоремы.

4 Семейства псевдослучайных функций

Определение 4.1. Семество функций $f_n : \{0,1\}^s \times \{0,1\}^{k(n)} \to \{0,1\}^{l(n)}$ (здесь n — параметр безопасноти, как и всегда, будем опускать его и писать $f_s(x) = f_{|x|}(s, x)$), которое удовлетворяет следующим условиям:

1. Семество должно быть малого размера, т.е. $s = poly(n)$ и тогда размер семества будет $2^s = 2^{poly(n)}$. Данный размер называют малым, так как всего функций $\{0,1\}^{k(n)} \to \{0,1\}^{l(n)}$ — $2^{l(n)2^{k(n)}}$ — двойная экспонента.
2. $f(s, x)$ должно вычисляться за полиномиальное время
3. Одно из свойств псевдослучайности:
   - ▷ **Слабая псевдослучайность**: $\forall m = poly(n) \forall (x_1, \dots x_m)$ величины $(f_s(x_1), \dots f_s(x_m))$ и $(g(x_1), \dots g(x_m))$ вычислительно неотличимы, где случайность берется по g — случайной функции (вообще любой случайной функции $\{0,1\}^{k(n)} \to \{0,1\}^{l(n)}$), а f — случайная из семества.
   - ▷ **Сильная псевдослучайность**: Адаптивный полиномиальный отличитель сначала запрашивает $f(x_1)$, потом вычисляет x_2, запрашивает $f(x_2)$ и т.д., в конце возвращает 0 или 1. Соответственно, мы будем трабовать, чтобы $\Pr\{D^{f_s} = 1\} \approx \Pr\{D^g = 1\}$ (отличаются меньше любого обратного полинома начиная с какого-то номера).

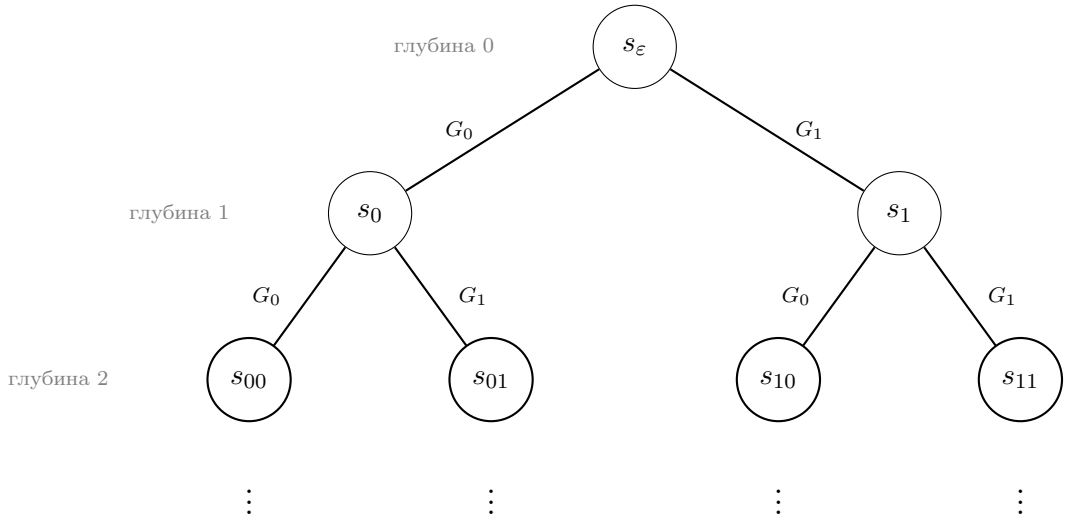
Пример (Слабая псевдослучайность выполнена, а первые два свойства — нет)**.** Рассмотрим функцию $f : \{0,1\}^{k(n)} \to \{0,1\}^{l(n)=k(n)}$, определяемую следующим правилом:

$$f(x) = \begin{cases} \text{случайное } v, x = 0 \\ 0, x = v \\ \text{случайное}, x \notin \{0, v\} \end{cases}$$

Заметим, что такое семество функций не удовлетворяет первому свойству (таких функций двойная экспонента). При этом $f(f(0)) = 0$, поэтому таких функций среди всех $\{0,1\}^{k(n)} \to \{0,1\}^{l(n)}$ экспоненциально мало.

4.1 Конструкция ПСФ из ГПСЧ

Пусть нам дан генератор G, удваивающий длину слова, т.е. $G(s) = G_0(s)G_1(s)$, где $|s| = |G_0(s)| = |G_1(s)|$. Построим бинарное дерево следующим образом: возьмем $s_\varepsilon = s$, и определим $s_{w0} = G_0(s), s_{w1} = G_1(s)$ (здесь $\varepsilon, w \in \{0,1\}^*$). Проще говоря, вычисляем $G(s)$ и каждый раз делим слово пополам, повторяя операцию:



Положим $f_s(x) = s_x$, где $|s| = |x|$. Проверим, что полученное семество действительно будет ПСФ:

1. $|s| = |x| = poly(|x|)$
2. $f_s(x)$ нетрудно вычислить за полином: действительно, нам не нужно каждый раз строить все дерево, достаточно каждый раз построить нужную нам ветку (соответствующую слову x).
3. **Слабая псевдослучайность**. Докажем, что $f_s(x)$ неотличима от случайного распределения на $\{0,1\}^n$ для одного x. Для доказательства, напишем в i-ом уровне дерева вместо $s_y, |y| = i$ истинно случайные биты r_y (заменим один слой на истинно случайные биты $|r_y| = |x|$ и пересчитаем все дерево ниже). Рассмотрим распределения, определенные следующим образом:

$$f^i_{\vec{r}}(x) = G_{x_n}(G_{x_{n-1}}(\dots(G_{x_{i+1}}(r_{x_1x_2\dots x_i}))))$$

Заметим, что f^0 — исходное семество псевдослучайных величин, а f^n — истинно случайные биты, при этом $f^i \sim f^{i+1}$. Тогда аналогично доказательству в теореме о ГПСЧ из односторонней перестановки, получаем, что $f^0 \sim f^n$.

Для нескольких слов достаточно проделать эту операцию не с одним словом, а с кортежом $(x_1, \dots x_m)$.

Заметим, что заменить m на экспоненту не получится из-за определения вычислительной неотличимости

5 Задача шифрования

Здесь мы переходим к части курса, где мы хотим, чтобы «те, кто нужно получили то, что нужно, а все остальные ничего не получили». В задаче шифрования приняты следующие обозначения (в дальнейшем будем обозначать их первыми буквами для удобства.):

1. Отправитель (Sender, Encoder) или Алиса (Alice) — имеет сообщение m и хочет его отправить его зашифрованную версию получателю
2. Получатель (Reciever, Decoder) или Боб (Bob) — хочет получить сообщение m' от отправителя, причем желательно, чтобы $m' = m$.
3. Взломщик (Adversary) или Ева (Eve) — хочет получить m.

Мы будем считать, что все трое знают алгоритмы друг друга (следует из формальной модели). При этом, мы не просто хотим, чтобы взломщик не узнал исходное сообщение, но и любую информацию, связанную с ним (кроме длины, так как сообщение может быть разбито на блоки).

Заметим, что в случае, когда взломщик не может угадать сообщение, получатель должен обладать некоторым секретом d, которого нет у взломщика. Исходя из этой идеи, рассмотрим два варианта шифрования:

1. **Криптография с закрытым ключом**. S тоже получает d и шифрует при помощи него сообщения. Формально:
   - ▷ m — исходное сообщние — знает только отправитель
   - ▷ d — закрытый ключ — знают только получатель и отправитель
   - ▷ $c = E(d, m)$ — зашифрованное сообщение — знают все
   - ▷ $m' = D(d, c)$ — расшифрованное сообщение
2. **Криптография с открытым ключом**. S и A оба получают открытый ключ e, парный к d, при помощи которого можно шифровать сообщения. Формально:
   - ▷ m — исходное сообщние — знает только отправитель
   - ▷ d — закрытый ключ — знает только получатель
   - ▷ e — открытый ключ — знает отправитель и взломщик (генерируется парно к d).
   - ▷ $c = E(e, m)$ — зашифрованное сообщение — знают все
   - ▷ $m' = D(d, c)$ — расшифрованное сообщение

Напоминание (Гаммирование)**.** Рассмотрим $S(m, d) = m \oplus d, R(c, d)$. Данный протокол никак не старает от проблемы $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ и экспоненциального противника. Однако у него все равно есть проблемы:

1. Он одноразовый, т.к. $(m_1 \oplus d) \oplus (m_2 \oplus d)$ разглашает $m_1 \oplus m_2$, что может являться чувствительной информацией.
2. Длина ключа равна длине сообщения.

5.1 Модели атак

5.1.1 Схемы, надежные относительно одноразового подслушивания

Определение 5.1. Схема, надежная относительно одноразового подслушивания — это такая схема шифрования, удовлетворяющая следующему свойству:

1. **Для криптографии с закрытым ключом:** $E(d, m_1) \sim E(d, m_2)$ при всех m_1, m_2 и случайном d.

2. **Для криптографии с открытым ключом:**. $(e, E(e, m_1)) \sim (e, E(e, m_2))$ при всех m_1, m_2 и случайном e, а также битам, используемым в E.

Следствие. В случае криптографии с открытым ключом, E ***обязано*** использовать другие случаные биты, кроме самого e. Действительно, иначе $E(e, m_1)$ однозначно восстановится по e, m_1, и тогда, например, взломщик сможет узнать, было ли отправлено именно сообщение m_1 или нет — это является чувствительной информацией.

Пример (Закрытый ключ на базе ГПСЧ)**.** Рассмотрим d — закрытая строка. Пусть G — некоторый ГПСЧ. Тогда положим $E(d, x) = D(d, x) = x \oplus G(d)$. Так как d неотличимо от случайных битов, то $E(d, x), D(d, x)$ — тоже. Но $D(d, E(d, x)) = x$ — то есть по ключу d можно восстановить исходное сообщение.

Определение 5.2 (Трудный бит для перестановки с секретом)**.** Пусть $f_e : D_e \to D_e$ — односторонняя перестановка с секретом (открытый ключ e, секрет d). Пусть $h_e : D_e \to \{0,1\}$ — полиномиально вычислимая функция. Говорим, что h_e является *трудным битом* для f_e, если выполнены два условия:

1. **Трудность без секрета.** Для любого полинома $q(\cdot)$ и любого противника P (полиномиального вероятностного алгоритма или семейства схем полиномиального размера)

$$\exists N\ \forall n > N: \quad \left| \Pr_{e,x}\{P(e, f_e(x)) = h_e(x)\} - \frac{1}{2} \right| \leqslant \frac{1}{q(n)},$$

   где e генерируется алгоритмом G, а x выбирается сэмплером $S(e)$ (почти равномерно из D_e).

2. **Лёгкость со секретом.** Существует полиномиальный алгоритм B_h, который по (e,d,y) вычисляет $h_e(f_e^{-1}(y))$, т.е.

$$B_h(e,d,f_e(x)) = h_e(x)$$

   для всех $x \in D_e$. Иными словами, зная секрет d, значение $h_e(x)$ легко восстановить по $f_e(x)$.

Пример (Открытый ключ на базе перестановки с секретом с трудным битом)**.** Сначала поймём, как шифровать один бит. Пусть $f_e(x)$ — перестановка с секретом, а $h_e(x)$ — трудный бит для неё. Значение $h_e(x)$ легко восстановить по $f_e(x)$, если знать секрет d: сначала считаем $x = f_e^{-1}(y)$, затем $h_e(x)$. Положим

$$E(e,b) = \big(f_e(x), b \oplus h_e(x)\big),$$

где x выбирается сэмплером $S(e)$ из D_e, и

$$D\big(d,(y,\sigma)\big) = \sigma \oplus h_e\big(f_e^{-1}(y)\big),$$

где $f_e^{-1}(y)$ считается при помощи d.

Для шифрования нескольких битов $l = poly(n)$ достаточно взять один случайный $x \in D_e$ и рассмотреть итерации f_e:

$$E(e,b_1,\ldots,b_l) = \big(b_1 \oplus h_e(x), b_2 \oplus h_e(f_e(x)),\ \ldots,\ b_l \oplus h_e(f_e^{l-1}(x)), f_e^l(x)\big).$$

Обозначим

$$\sigma_i = b_i \oplus h_e(f_e^{i-1}(x)), \qquad y = f_e^l(x).$$

Тогда шифртекст имеет вид $(\sigma_1,\ldots,\sigma_l,y)$, а расшифрование:

$$D\big(d,(\sigma_1,\ldots,\sigma_l,y)\big) = \big(\sigma_1 \oplus h_e((f_e^{-1})^l(y)), \sigma_2 \oplus h_e((f_e^{-1})^{l-1}(y)),\ \ldots,\ \sigma_l \oplus h_e(f_e^{-1}(y))\big).$$

Стойкость доказывается аналогично теореме Блюма–Микали.

5.1.2 Схемы, надежные относительно многоразового подслушивания

Определение 5.3. Схема, надежная относительно одноразового подслушивания — это такая схема шифрования, удовлетворяющая следующему свойству (здесь $p = poly(n)$):

1. **Для криптографии с закрытым ключом:** $E(d, m_1, \ldots m_p) \sim E(d, s_1, \ldots s_p)$ при всех $(m_1, \ldots m_p), (s_1, \ldots s_p)$ и случайном d.
2. **Для криптографии с открытым ключом:**. $(e, E(e, m_1, \ldots m_p)) \sim (e, E(e, s_1, \ldots s_p))$ при всех $(m_1, \ldots m_p), (s_1, \ldots s_p)$ и случайном e, а также битам, используемым в E.

Замечание. В случае с открытым ключом определения схем, надежных относительно многоразового и одноразового подслушивания (т.к. случайность берется по битам, используемым в E и e), однако для закрытого ключа это неверно.

Пример (Закрытый ключ на базе ПСФ в слабом смысле)**.** Пусть f_r — семейство ПСФ, где r — ключ (номер функции). Положим $d = r$. Шифрование сообщения m:

$$E(r, m) = (x, m \oplus f_r(x)),$$

где x выбирается случайно (для каждого сообщения заново). Расшифрование:

$$D(r, (x, y)) = y \oplus f_r(x).$$

Стойкость относительно многоразового подслушивания следует из определения ПСФ в слабом смысле.

5.1.3 Атаки с выбором сообщения

До этого мы рассматривали только пассивного злоумышленника, который просто подслушивает. Однако на практике злоумышленник может быть гораздо активнее: например, он может притворяться получателем, отправлять отправителю свои сообщения и смотреть,

что тот ответит. Или он может заставить получателя расшифровать произвольные шифртексты (например, через ошибки в протоколе или через какой-нибудь оракул). В таких случаях говорят об атаках с выбором.

Интуитивно есть два типа таких атак:

1. **Chosen Plaintext Attack** (CPA, атака с выбором открытого текста). Злоумышленник может попросить зашифровать любые сообщения по своему выбору, а потом перехватывает целевой шифртекст. Он не должен отличить этот шифртекст от шифртекста другого сообщения.

2. **Chosen Ciphertext Attack** (CCA, атака с выбором шифртекста). Злоумышленник может попросить расшифровать любые шифртексты (кроме целевого), а потом перехватывает целевой шифртекст. Он снова не должен отличить его от шифртекста другого сообщения.

Пример (Закрытый ключ на базе ПСФ в сильном смысле)**.** Пусть f_r — семейство ПСФ в сильном смысле, где r — ключ (номер функции). Положим $d = r$. Шифрование сообщения m:

$$E(r, m) = (x, m \oplus f_r(x)),$$

где x выбирается случайно для каждого сообщения. Расшифрование:

$$D(r, (x, y)) = y \oplus f_r(x).$$

Так как f_r — ПСФ в сильном смысле, то даже при адаптивных запросах к f_r значения $f_r(x_i)$ для новых случайных x_i вычислительно неотличимы от случайных. Поэтому для любого m шифртекст $(x, m \oplus f_r(x))$ неотличим от $(x, m \oplus u)$, где u — случайный бит. Значит, никакой полиномиальный злоумышленник не может угадать, какое из двух сообщений m_0, m_1 было зашифровано, с заметным преимуществом. Иными словами, схема CPA-стойкая.

6 Протоколы привязки

Мы хотим понять, как двоим можно договориться о случайных битах, если, например, они не доверяют внешнему источнику информации. Существует два подхода к этой ситуации:

1. Алиса хочет получить 1, Боб хочет получить 0, соответственно, им нужно получить распределение, неотличимое от $Bern(1/2)$.

2. Один из них хочет получить случайный бит, а второй хочет сдвинуть распределение вероятностей в любую сторону

Оказывается, что на практике первый вариант возможен, а вот второй — нет. Рассмотрим, как можно добиться первого на языке физических аналогий:

1. Алиса выбирает случайное $a \in \{0, 1\}$, кладет его в шкатулку, закрывает на ключ и отправляет шкатулку (без ключа) Бобу.

2. Боб выбирает случайное $b \in \{0, 1\}$ и отправляет его Алисе.

3. После этого Алиса отправляет ключ от шкатулки, в итоге оба имеют a, b и результат: $a \oplus b$

В данной ситуации Алиса может сжульничать, например, так: сделать шкатулку с двойным дном. В таком случае, в зависимости от отправленного b, она может отправить ключ либо от 1, либо от 0 и сгенерировать таким образом тот бит, который она хочет. У боба же не должно быть возможности «просветить» шкатулку на рентгене и узнать, что лежит внутри, чтобы он не мог выбрать бит b неслучайно (исходя из значения в шкатулке). Далее мы будем называть, шкатулки «привязками».

Для доказательства работоспособности алгоритма, необходимо показать *корректность* (правильный ключ дает исходное сообщение), *непрозрачность* (либо вычислительная: привязки к 0 неотличимы от привязок к 1, либо абсолютная: привязки к 0 и привязки к 1 одинаково распределены) и *неподменяемость* (либо вычислительная: для одной привязки трудно найти ключи, которые дадут разные исходные биты, либо абсолютная: ни одно слово не является привязкой и к 0, и к 1)

Замечание. Таким образом, невозможно сразу же добиться абсолютной неподменяемости и абсолютной неотличимости.

6.1 Неинтерактивный протокол привязки

В таком случае будет два алгоритма: $C(b, k)$ — изготавливает привязку, $R(c, k)$ — «открывает» её, где k — строка, c — строка, $R(c, k) \in \{0, 1, \perp\}$. Формально, условия выше можно переписать следующим образом:

Определение 6.1. Пара алгоритмов (C, R) называется неинтерактивным протоколом привязки, если выполнены следующие свойства:

1. *Корректность*: $\Pr\{R(C(b, k), k) = b\} \approx 1$ (мы допускаем, что может случиться что-то плохое).

2. *Абсолютная неподменяемость*: $\nexists c, k_0, k_1 : R(c, k_0) = 0 \wedge R(c, k_1) = 1$ или, что равносильно: $\forall b, k, k' : R(C(b, k), k') \in \{b, \perp\}$.

3. *Вычислительная непрозрачность* $C(0, k) \sim C(1, k)$

Теорема 6.1. *Если существует односторонняя перестановка, то существует неинтерактивный протокол привязки*

Доказательство. Нам известно, что если существует односторонняя перестановка, то существует и односторонняя перестановка g с трудным битом h. Пусть $k = x$ — случайный элемент из области определения g, и положим (здесь $|C_1| = n, |C_2| = 1$)

$$C(b, k) = (g(k), b \oplus h(k))$$

$$R((C_1, C_2), k) = \begin{cases} c_2 \oplus h(k), c_1 = g(k) \\ \perp, \text{иначе} \end{cases}$$

1. *Корректность*: очевидна (более того, вероятность корректно все расшифровать в точности равна 1).

2. *Абсолютная неподменяемость*: Если $C_1 = g(k_0), C_1 = g(k_1)$, то $k_0 = k_1 \Rightarrow C_2 \oplus h(k_0) = C_2 \oplus h(k_1)$

3. *Вычислительная непрозрачность* $(g(k), h(k)) \sim (g(k), r)$, но тогда $(g(k), b \oplus h(k)) \sim (g(k), b \oplus r) \sim (g(k), r)$.

□

6.2 Интерактивный протокол привязки

Здесь будем подразумевать, что между отправителем S и получателем R происходит диалог, в процессе которого отправитель получает ключ $k = S^R(b)$ (канонический вариант — все случайные биты, которые он использовал), а получатель — привязку $c = R^S(b)$. После этого, после отправки ключа k, получатель применяет алгоритм $T(k, c)$ и получает b (или выдает ошибку $\perp_S, \perp_R$ — с чьей стороны она произошла).

Определение 6.2. Пара полиномиально вероятностных алгоритмов (S, R, T) называется интерактивным протоколом привязки, если огни удовлетворяют следующим условиям:

1. *Корректность*: $\Pr\{T(c, k) = b\} \approx 1$.

2. *Вычислительная неподменяемость*: $\Pr\{\exists k_0, k_1 : (T(c, k_0) = 0 \wedge T(c, k_1) = 1)\} \approx 0$

3. *Вычислительная непрозрачность* определяется по аналогии с вычислительно нулевым разглашением. Получатель должен уметь сгенерировать распределение, вычислительно неотличимое от $VIEW_R^{S(b)}$ (т.е. диалог между ним и отправителем)

Утверждение 6.1. *Если существует ГПСЧ, то существует интерактивный протокол привязки*

Доказательство. Возьмем ГПСЧ $G : \{0,1\}^n \to \{0,1\}^{3n}$. Рассмотрим следующий диалог:

1. Получатель отправляет отправителю $r \in \{0,1\}^n$ — случайные биты

2. Отправитель отправляет $s = G(k) \oplus r$, если $b = 1$ или $s = G(k)$, если $b = 0$.

3. После этого отправляет ключ k, отправителю остается проверить, чему равно b, в зависимости от привязки (r, s), по есть:

$$T((r, s), k) = \begin{cases} 0, s = G(k) \\ 1, s = G(k) \oplus r \\ \perp_S, \text{иначе} \end{cases}$$

Докажем, что этот протокол удовлетворяет условиям интерактивного протокола привязки:

1. *Корректность*: работает всегда, кроме $r = 0^{3n}$.

2. *Вычислительная неподменяемость*: Если можно неоднозначно раскрыть привязку, то $G(k_0) = G(k_1) \oplus r \Rightarrow r = G(k_0) \oplus G(k_1)$. Тогда всего вариантов для $r \leqslant 2^n$, при этом всевозможных r ровно 2^{3n}. Получили, что вероятность получить такое r, что при нем возможно неоднозначная расшифровка привязки $= \frac{1}{2^n} \approx 0$.

3. *Вычислительная непрозрачность* из определения ГПСЧ получаем, что s вычислительно неотличим от случайных битов

□

7 Протоколы идентификации

Пусть задача состоит в том, что нам нужно авторизоваться на сервере, т.е. доказать свою идентичность. Соответственно, будем иметь прувера P (клиент) и верификатора V (сервер), и после некоторого общения верификатор должен выдать ответ «да» или «нет». Можно сформулировать это так: на этапе регистрации при помощи генератора K прувер получает x, а верификтор y и задача прувера состоит в том, чтобы доказать верификатору, что тот знает x. Как и прежде, здесь будет протокол с *закрытым ключом*, т.е. в случае, когда $x = y$ и с *открытым ключом*, т.е. $x \neq y$.

Определение 7.1. Будем считать протокол идентификации надежным, если выполнены следующие требования:

1. *Корректность*: $\Pr\{V^{P(x)}(y) = 1\} \approx 1$
2. *Надежность*: будем рассматривать надежность относительно различных атак.

Определение 7.2. *Протокол, надежный относительно простой атаки* — взломщик просто приходит и пытается залогиниться на сервер.

1. Для закрытого ключа: $\forall P^* : \Pr\{V^{P^*}(y) = 1\} \approx 0$
2. Для открытого: $\forall P^* : \Pr\{V^{P^*(y)}(y) = 1\} \approx 0$.

Пример. Для закрытого ключа достаточно положить $K : x = y$ — равномерно распределенные по строкам длины n. И тогда прувер просто сообщает верификатору x, последний проверяет равенство с y.

Для открытого ключа достаточно положить x — случайное, $y = f(x)$, где f — односторонняя. Соответственно, верификатор будет проверять, что $f(x) = y$.

Заметим однако, что данные атаки будут не надежными относительно следующей модели атак:

Определение 7.3. *Протокол, надежный относительно атаки с подслушиванием* — P^* получает протокол успешной авторизации (и открытый ключ), после чего пытается авторизоваться.

Пример. Закрытый ключ реализуется на базе ПСФ: $x = t =$ номер функции из семейства. Соответственно, верификатор посылает пруверу v, а прувер присылает назад w. В таком случае верификатор проверяет $w = f_t(v)$. Так как $f_t(v)$ распределено равномерно, если не знать t, то взломщик не сможет узнать, какое w отправить, если не знать t.

Протокол для открытого ключа реализуется на базе односторонних перестановках с секретом или протоколов шифрования с открытым ключом. Пример для протоколов шифрования: отправляем пруверу w, зашифрованное при помощи e, он возвращает нам расшифрованное при помощи d исходное сообщение.

Определение 7.4. *Протокол, надежный относительно фишинговой атаки* — V^* проводит $poly(n)$ диалогов с $P(x)$, после чего пытается авторизоваться на настоящем V. Соответственно, вероятность успешного взлома должна быть пренебрежимо мала.

Пример. Для закрытого ключа будет работать аналог, только с семейством ПСФ в сильном смысле. Для открытого же ключа предыдущая схема может «сломаться»: взломщик может специально подобрать такие сообщения, чтобы узнать некоторую информацию про открытый ключ. Поэтому надо действовать иначе. Здесь нам поможет *доказательство знания с нулевым разглашением.*

7.1 Доказательство знания с нулевым разглашением

Напоминание. $\mathsf{GI} \in \mathbf{CZK}$

Доказательство. Прувер знает $\varphi(G_0) = G_1$ и хочет доказать верификатору, что $G_0 \cong G_1$. Прувер отправляет верификатору $\sigma(G_0)$ для случайного σ, тот возвращает $b \in \{0, 1\}$, после чего прувер отправляет τ — изоморфизм между $G_b, \sigma(G_0)$. □

Что в данном случае означает «знание» прувером истинной перестановки φ? Ровно то, что он, даже если она не задана явно, может отвечать так, что их его ответов будет следовать, что он действительно обладает некоторой информацией о φ. Теперь сформулируем это формально:

Определение 7.5. Пусть $R(x, y)$ — полиномиальный предикат. Будем говорить, что протокол (P, V) — доказательство знания с нулевым разглашением, если

1. $\Pr\{V^{P(y)}(x) = 1\} > \frac{2}{3}$, если $R(x, y) = 1$
2. $\neg\exists y : R(x, y) = 1 \Rightarrow \forall P^* : \Pr(V^{P^*}(x) = 1) < \frac{1}{3}$
3. $\forall V^* \exists M^* : M^*(x) \sim view_{V^*}^{P(y)}(x)$ при любом y, таком, что $R(x, y) = 1$
4. $\exists E \forall P^* (\Pr\{V^{P^*}(x) = 1\} > \frac{2}{3} \Rightarrow \Pr\{E^{P^*}(x) = y$, т.ч. $R(x, y) = 1\} > \frac{2}{3})$, причем E во втором случае общается с P^* как с оракулом (т.е. P^* может ответить на один и тот же вопрос несколько раз и т.д.).

Теорема 7.1 (б/д)**.** *Если существует односторонняя функция, и $R(x, y)$ полиномиально, то $\exists$ протокол доказательства знания с вычислительно нулевым разглашением.*

8 Задача аутентификации

Пусть имеется клиент и сервер, которые отправляют друг другу некоторые сообщения, причем в конце сервер выдает «да» или «нет» (знает ли пользователь пароль или нет).

Здесь также существуют аутентификации разных типов:

1. **Аутентификация с закрытым ключом:** и клиент и сервер знают пароль. Однако такая схема считается ненадежной, т.к. в таком случае пароль можно украсть с сайта.
2. **Криптография с открытым ключом:** только клиент знает пароль, а на сервере хранится хэш/еще что-то подобное, что не позволяет злоумышленнику, взломавшему сервер украть пароль.

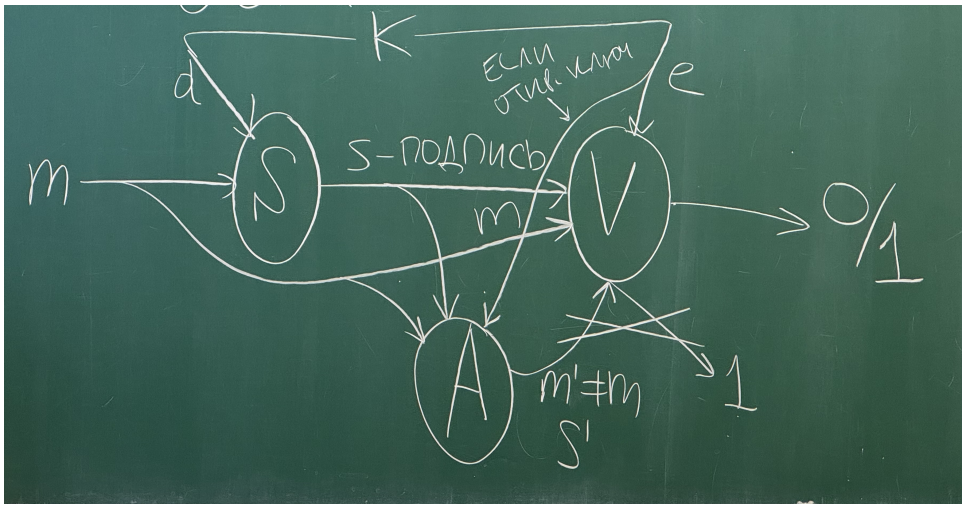
Для задачи аутентификации существют различные типы атак:

1. Простая — злоумышленник пытается аутентифицироваться на сервере
2. Однократная прослушка — злоумышленник слушает одно сообщение и пытается аутентифицироваться
3. Многократная прослушка — злоумышленник слушает несколько (полиномиальное число) сообщений и пытается аутентифицироваться
4. Фишинг — злоумышленник притворяется сервером (например, подделывает страницу логина)
5. Man-in-the-middle attack (человек посередине) — злоумышленник играет роль «прокси» между сервером и клиентом, пересылая сообщения одного другому.

9 Цифровая подпись

В реальной жизни подписи существуют для доказательства подлинности того или иного документа. Нетрудно понять, что у физических подписей имеется огромное количество недостатков: например, их легко копировать, так как они не привязаны к тексту документа. Конечно же, с этим пытаются бороться, но идея остается: так как подписи очень легко копировать, они обязательно должны зависеть от сообщения.

Соответственно, модель цифровой подписи выглядит следующим образом. Пусть S — подписывающая сторона, V — проверяющая сторона. Мы имеем генератор K ключей d, e — закрытого (есть только у S) и открытого (в случае открытого ключа есть у всех, иначе — только у V). Пусть имеется сообщение $m, s = S(m)$ — его подписью. Соответственно, верификатор выдает 0 или 1 — действительно ли S подписал это сообщение.



Модели атак в данной модели следующие:

1. Однократная атака — Алиса подслушивает один раз, после чего пытается подписать хотя бы какое-то сообщение
2. Многократная атака — Алиса подслушивает несколько раз, после чего пытается подписать хотя бы какое-то сообщение
3. Атака с выбором сообщения — Алиса произвольно может подписывать любые сообщения, после чего пытается подписать свое сообщение

9.1 Протокол одноразовой подписи 1 бита с открытым ключом

Для протокола будем требовать, чтобы K, S, V были полиномиально вероятностными. Условия для стойкости протокола определяются следующим образом:

1. *Корректность*: $\Pr\{V(e, S(d, b), b) = 1\} \approx 1$
2. *Защита от однократной атаки*: $\forall b \forall \{D_n\} \Pr\{V(e, D_n(e, S(d, b), b), 1 - b) = 1\} \approx 0$

Пример (Протокол на базе односторонней функции)**.** Пусть f — односторонняя функция. Рассмотрим $d = (x_0, x_1), e = (y_0, y_1), y_b = f(x_b)$. Соответственно, $S(d, b) = x_b, V(e, s, b) = \begin{cases} 1, f(s) = y_b \\ 0, \text{иначе} \end{cases}$. Нетрудно видеть, что если D_n смог подписать некоторый текст, то в таком случае он умеет обращать f с не меншьей вероятностью.

9.2 Протоколы одноразовой подписи с открытым ключом

Определения аналогичны случаю с одним битом, поэтому перейдем сразу же к самому протоколу:

Пример (Протокол подписи k битов, устойчивый относительно атаки с выбором одного сообщения)**.** Здесь понимаем k как величину, не зависящую от n — параметра безопасности. Здесь просто рассмотрим k протоколов для однобитовой подписи, как было описано выше, генерированных независимо друг от друга. Доказательство корректности примерно следующее: если взломщик с существенной вероятностью взламывает подпись, то он гарантированно делает это с существенной вероятностью для какого-то фиксированного бита. В таком случае, с не меншьей вероятностью он умеет обращать f, пришли к противоречию.

Замечание. Относительно выбора двух сообщений данный протокол не будет устойчив: действительно, если попросить подписать сообщения $0 \dots 0, 1 \dots 1$, то мы узнаем полностью закрытый ключ и сможем подписывать любые сообщения.

Для дальнейших протоколов понадобится такой объект как *хеш-функция*. Смысл хеш-функции в том, что он должен позволять делать следующее: например, сегодня Боб сообщает $f(x)$, а завтра — x, после чего мы понимаем, что он действительно хотел сказать x. Кажется, что это можно реализовать обычными односторонними функциями, но что, если он смог сгенерировать целый набор $x_1, \dots x_n$ с совпадающими образами при действии f, а на следующий день выбрал наиболее выгодное для себя x_i? Поэтому кроме односторонности, хеш функция должна обладать свойством трудного нахождения коллизий. Таким образом мы приходим к следующему определению:

Определение 9.1. Будет на следующей лекции

Пример (Протокол одноразоой подписи слова произвольной длины)**.** Рассмотрим семейство хеш-функций $h : \{0,1\}^{m(n)} \times \{0,1\}^* \to \{0,1\}^k$. Соответственно, протокол будет следующий: сначала хешируем сообщение, а потом подписываем хеш при помощи предыдущих протоколов. Для того, чтобы взломщик мог сломать этот алгоритм, ему нужно либо найти другое слово с таким же хешем, либо подписать другой хеш — в обоих случаях вероятность пренебрежимо мала. Важно: здесь индекс функции будет частью обоих ключей.

Также приведем еще один протокол подписи слова произвольной длины, которое построено по принципу Блокчейна:

Пример (Протокол одноразовой подписи слова произвольной длины (многоразовый в случае сохранения памяти между сессиями))**.** Будем вместе с сообщением передавать еще и новый сгенерированный открытый ключ для следующего сообщения (закрытый ключ при этом храним в памяти подписывающей стороны). То есть сначала генерируем пару (d_0, e_0) и публикуем e_0. Для подписи m_1 генерируем (d_1, e_1) и отправляем $s_1 = S_{d_0}(h(m_1\|e_1))$ вместе с e_1. Для m_2 уже используем d_1: $s_2 = S_{d_1}(h(m_2\|e_2))$, и так далее. Получается цепочка ключей, подписанных нами. Проверяющий идет по цепочке: проверяет подпись относительно текущего открытого ключа и переходит к следующему. Важно: старые закрытые ключи больше не используем, иначе одноразовость ломается.

Пример (Протокол одноразовой подписи слова произвольной длины на двоичном дереве открытых ключей)**.** В листьях двоичного дерева храним пары (d_i, e_i) для одноразовой подписи. Внутреннюю вершину строим как хеш от детей: $u = h(u_L\|u_R)$, а в корне получаем открытый ключ e, который и публикуем. Чтобы подписать сообщение m в листе i, передаем открытый ключ e_i, путь от e_i до корня и одноразовую подпись $s = S_{d_i}(m)$. Проверяющий по e_i и пути восстанавливает корень и сравнивает с e, после чего проверяет s. Важно: каждый лист используется только один раз, а дерево можно строить заранее на 2^t сообщений.